

NAME-TIASA JANA

EMP ID-7170

CAPSTONE PROJ-1

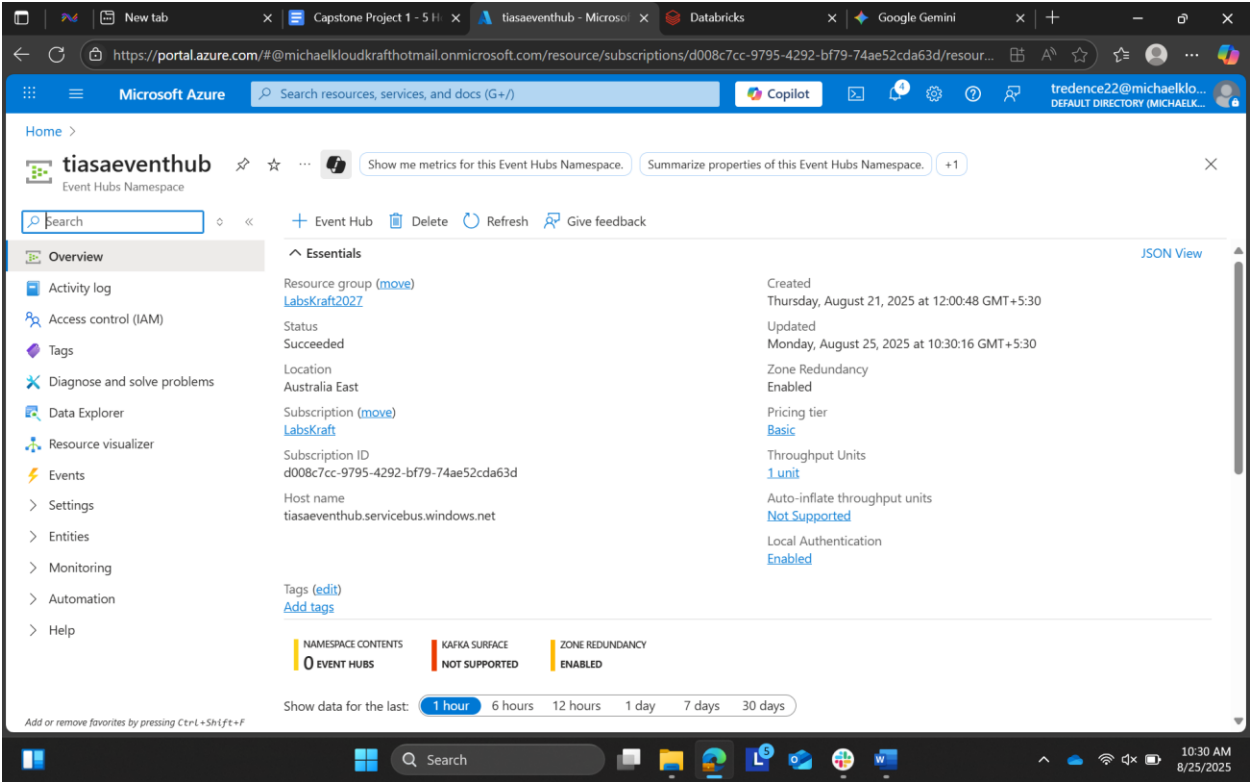
1) Creation of storage account and uploading file under container

The screenshot shows the Microsoft Azure portal interface. The main heading is "tiasastore_1756097594779 | Overview". Below this, there's a search bar and a list of actions: Delete, Cancel, Redeploy, Download, and Refresh. The "Overview" tab is selected, showing a green checkmark and the text "Your deployment is complete". The deployment details include: Deployment name: tiasastore_1756097594779, Subscription: LabsKraft, Resource group: LabsKraft2027, Start time: 8/25/2025, 10:23:55 AM, and Correlation ID: 3adbcebf-2c48-42be-aa49-c2c0f3fd12a. A "Go to resource" button is present. On the right, there's a "Deployment succeeded" notification: "Deployment 'tiasastore_1756097594779' to resource group 'LabsKraft2027' was successful." Below this, there are sections for "Cost Management", "Microsoft Defender for Cloud", "Free Microsoft tutorials", and "Work with an expert".

The screenshot shows the Microsoft Azure portal interface for the "tia" container. The main heading is "tia | Overview". Below this, there's a search bar and a list of actions: Add Directory, Upload, Refresh, Delete, Copy, Paste, Rename, Acquire lease, Break lease, and Edit columns. The "Overview" tab is selected, showing the "Authentication method: Access key (Switch to Microsoft Entra user account)". Below this, there's a search bar for "Search blobs by prefix (case-sensitive)" and a dropdown for "Only show active objects". The "Showing all 1 items" section displays a table with the following data:

Name	Last modified	Access tier	Blob type	Size	Lease state
retail_iot_data.csv	8/25/2025, 10:26:13 AM	Hot (Inferred)	Block blob	130.76 KiB	Available

Created namespace for eventhub



Event Hubs (1)

Search to filter items by name...			
Name	Status	Message retention	Partition count
eventhubtiasa	Active	1 hour	1

Validation failed for SQL database

The screenshot shows the Microsoft Azure portal interface. The main content area is titled 'Create SQL Database' and includes a 'Validation failed' message: 'Validation failed. Click here to view details. →'. Below this, there are tabs for 'Basics', 'Networking', 'Security', 'Additional settings', 'Tags', and 'Review + create'. The 'Review + create' tab is selected. On the right side, there is an 'Errors' panel with a 'Summary' tab. The error message states: 'Resource 'tiasa_db' was disallowed by policy. (Code: RequestDisallowedByPolicy)'. The policy is identified as 'Restrict_SQL_basic'. Below the error message, there are links for 'Check Usage + Quota' and 'New Support Request'. The bottom of the screen shows the Windows taskbar with the time 10:39 AM on 8/25/2025.

Synapse Workspace Created

The screenshot shows the Microsoft Azure portal interface for a Synapse Analytics deployment. The main content area is titled 'Microsoft.Azure.SynapseAnalytics-20250825104117 | Overview'. It shows a 'Deployment' status of 'Succeeded'. A message states: 'Your deployment is complete'. Below this, there are details for the deployment: 'Deployment name: Microsoft.Azure.SynapseAnaly...', 'Subscription: LabsKraft', 'Resource group: LabsKraft2027', 'Start time: 8/25/2025, 10:45:32 AM', and 'Correlation ID: b66185fa-8386-409d-b513-40...'. There is a 'Go to resource group' button. On the right side, there is a 'Deployment succeeded' notification: 'Deployment 'Microsoft.Azure.SynapseAnalytics-20250825104117' to resource group 'LabsKraft2027' was successful.' Below this, there are links for 'Pin to dashbo...' and 'Go to resource gr...'. The bottom of the screen shows the Windows taskbar with the time 10:49 AM on 8/25/2025.

AccessConnector Deployed

AccessConnectorCreate-20250825105515 | Overview

Deployment

Search

Delete Cancel Redeploy Download Refresh

Overview

Inputs

Outputs

Template

✓ Your deployment is complete

Deployment name : AccessConnectorCreate-20250...

Subscription : LabsKraft

Resource group : LabsKraft2027

Start time : 8/25/2025, 10:56:25 AM

Correlation ID : 4570c87b-8273-45c9-8fd9-e3...

Deployment details

Resource	Type	Status	Operatic
tiasaconnect	Access Connector for Azure Dat	Created	Operatic

Next steps

Go to resource

Cost management

Get notified to stay within your budget and prevent unexpected charges on your bill.

Set up cost alerts >

Microsoft Defender for Cloud

Secure your apps and infrastructure

Go to Microsoft Defender for Cloud >

Free Microsoft tutorials

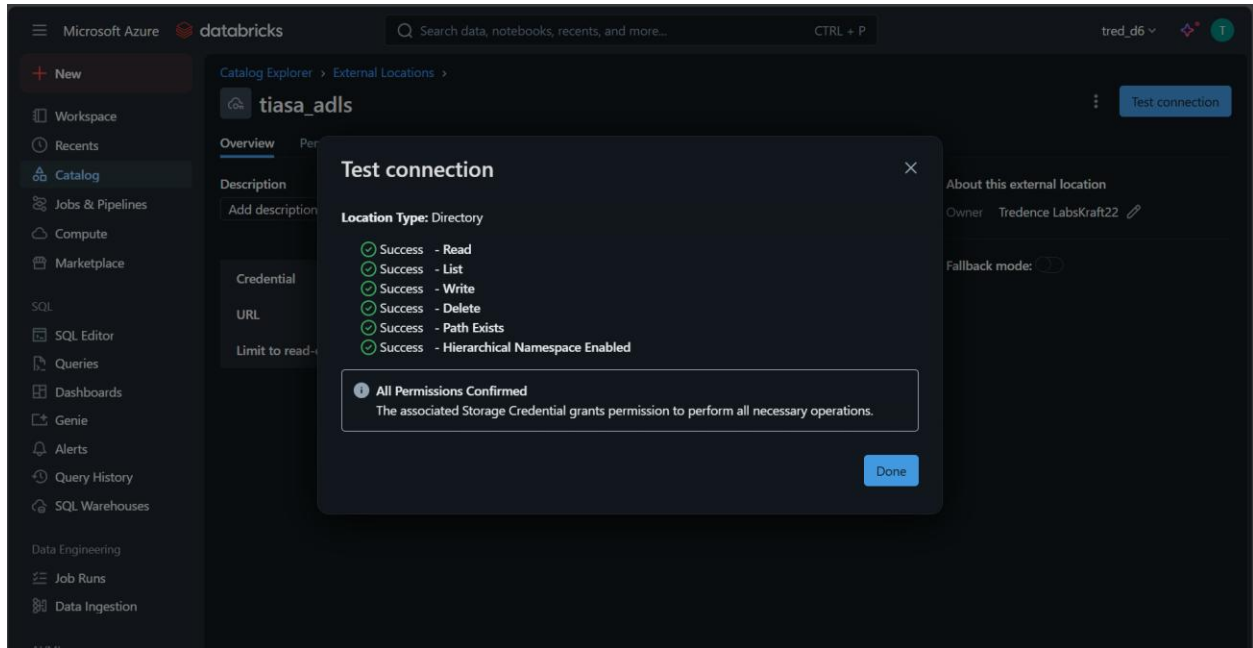
Start learning today >

Work with an expert

Gave Storage blob contributor and connector to azure connector

Storage Blob Data Contributor (2)					
☐	 tiasasynapse 27430e0d-80ef-4d...	Managed identity	Storage Blob Data Contrib...	 This resource	Add
☐	 Tredence LabsKraft22 2a23a83d-d25a-498...	User	Storage Blob Data Contrib...	 This resource	Add
Storage Blob Delegator (1)					
☐	 tiasaconnect d834356b-536a-44...	Managed identity	Storage Blob Delegator	 This resource	None
Contributor (3)					
☐	 tiasaconnect d834356b-536a-44...	Managed identity	Contributor	 This resource	None
☐	 vikram				

Created External Location for Unity Catalog



Created Unity Catalog

The screenshot shows the Databricks Catalog Explorer interface for a catalog named 'tiasacatalog3'. The interface is dark-themed. At the top, there's a breadcrumb 'Catalog Explorer >' and the catalog name 'tiasacatalog3' with a copy icon and a star icon. To the right are 'Share' and 'Create schema' buttons. Below this is a tab bar with 'Overview', 'Details', 'Permissions', and 'Workspaces'. The 'Overview' tab is active. Under 'Description', there are 'AI generate' and 'Add' buttons. A search bar 'Filter schemas' shows '2 schemas'. Below is a table with columns 'Name', 'Owner', and 'Created at'. The table lists two schemas: 'default' owned by 'tredence22@michaelcloudkraft...' and 'information_schema' owned by 'System user', both created on 'Aug 25, 2025, 11:16 AM'. Below the table is the 'About this catalog' section, showing the owner 'Tredence LabsKraft22' with an edit icon. At the bottom is a 'Tags' section with an 'Add tags' button.

Catalog Explorer >

tiasacatalog3

Share Create schema

Overview Details Permissions Workspaces

Description

AI generate Add

Filter schemas 2 schemas

Name	Owner	Created at
default	tredence22@michaelcloudkraft...	Aug 25, 2025, 11:16 AM
information_schema	System user	Aug 25, 2025, 11:16 AM

About this catalog

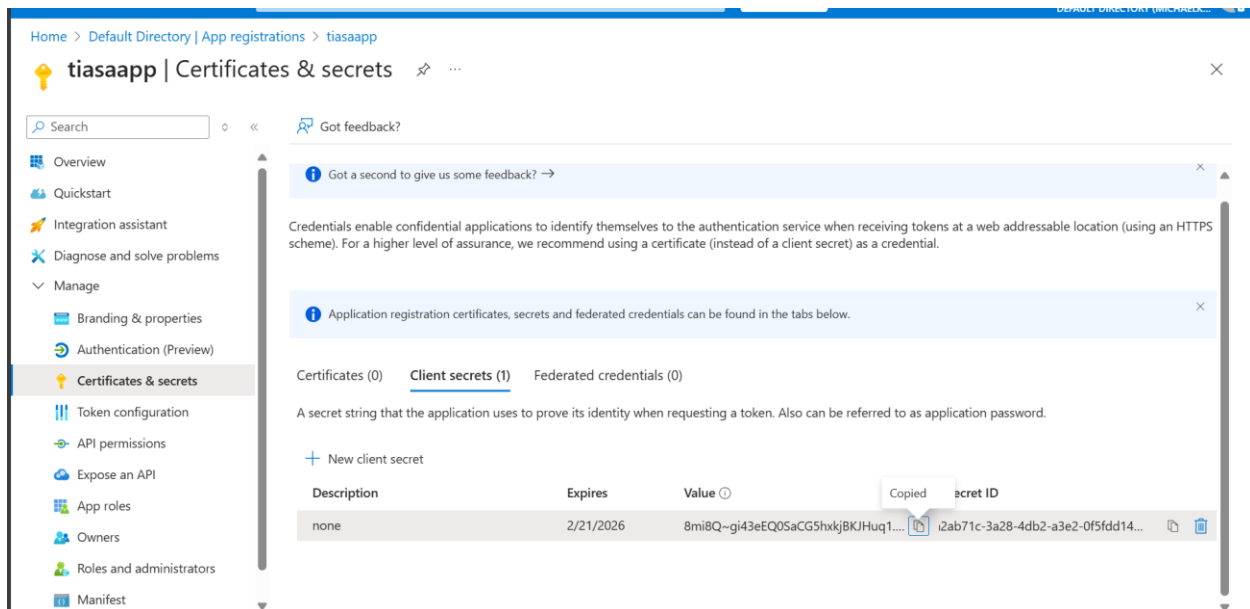
Owner Tredence LabsKraft22

Tags

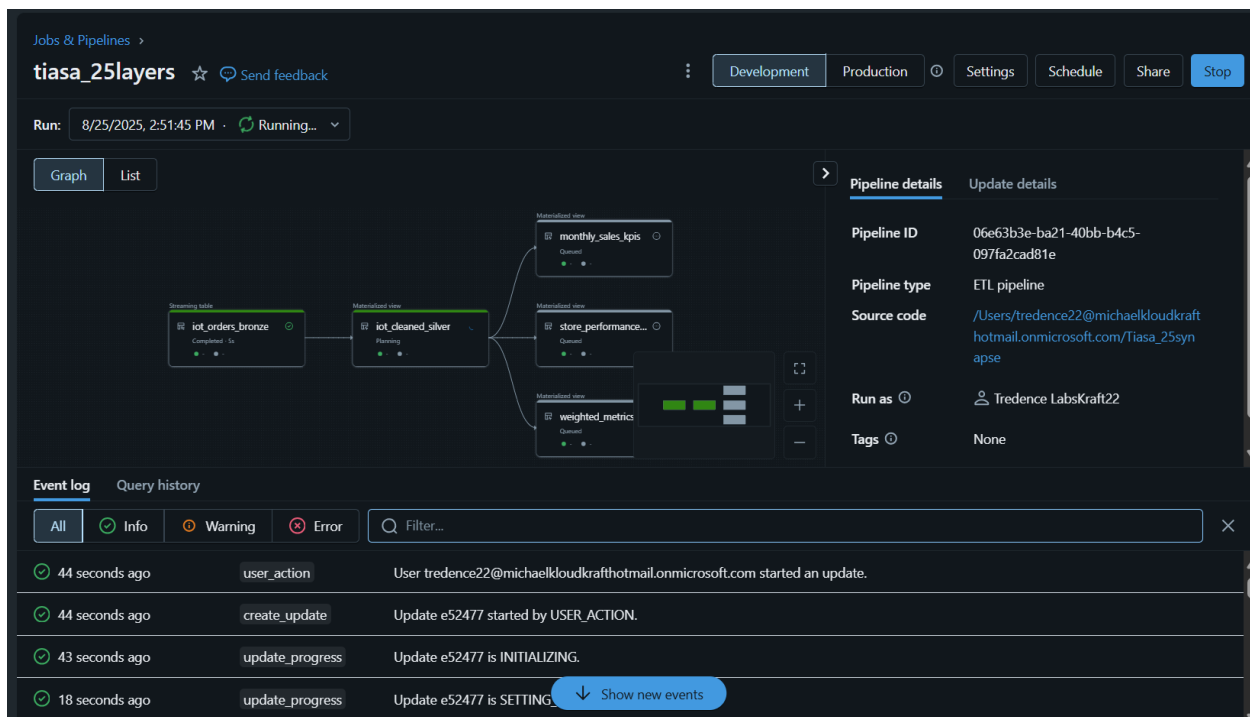
Add tags

For mounting ADLS to Databricks

***Went to Entra ID to register app (for client id,tenant id and client value)**



Deployed a pipeline for adls source



Couldnot implement POverBi as access was not provided for Tred6 workspace

Code for medallion architecture on adls storage data

```
import dlt

import pyspark.sql.functions as F

from pyspark.sql.types import StructType, StructField, StringType, IntegerType, DoubleType, TimestampType


# Define schema for the raw CSV data
iot_schema = StructType([
    StructField("OrderID", StringType(), True),
    StructField("CustomerID", StringType(), True),
    StructField("Region", StringType(), True),
    StructField("StoreID", StringType(), True),
    StructField("Product", StringType(), True),
    StructField("Quantity", IntegerType(), True),
    StructField("UnitPrice", DoubleType(), True),
    StructField("SalesAmount", DoubleType(), True),
    StructField("OrderTime", StringType(), True),
    StructField("DeviceType", StringType(), True),
    StructField("Temperature", DoubleType(), True),
    StructField("DeviceStatus", StringType(), True),
    StructField("AnomalyFlag", IntegerType(), True),
    StructField("AnomalyType", StringType(), True)
])


# -----
# Bronze Layer: Ingestion (Raw Data)
# -----

# This table ingests the raw data stream using Auto Loader.

@dlt.table(
```



```

    name="tiasacatalog3.bronze.iot_orders_bronze",
    comment="Raw ingested data from ADLS."
)

def iot_orders_bronze():
    # Use spark.readStream with cloudFiles (Auto Loader) for a production-like streaming setup.
    return (
        spark.readStream
            .format("cloudFiles")
            .option("cloudFiles.format", "csv")
            .option("header", True)
            .schema(iot_schema)
            .load("abfss://tia@tiasastore.dfs.core.windows.net/retail_iot_data.csv")
    )

# -----
# Silver Layer: Transformation (Cleaned Data)
# -----
# This table cleans the Bronze data and performs type casting and deduplication.
@dlt.table(
    name="tiasacatalog3.silver.iot_cleaned_silver",
    comment="Cleaned and validated data with correct types and no duplicates.",
    table_properties={"delta.enableChangeDataFeed": "true"} # Enable Change Data Feed for upserts
)

def iot_cleaned_silver():
    # Read from the Bronze table using dlt.read()
    df_bronze = dlt.read("iot_orders_bronze")

    # Renaming columns for clarity and consistency

```

```

df_silver = df_bronze.toDF(
    "order_id", "customer_id", "region", "store_id", "product", "quantity",
    "unit_price", "sales_amount", "order_time", "device_type", "temperature",
    "device_status", "anomaly_flag", "anomaly_type"
)

# Apply data type conversions and transformations
df_silver = (df_silver
    .withColumn("order_time", F.to_timestamp(F.col("order_time")))
    .withColumn("sales_amount_calculated", F.col("quantity") * F.col("unit_price"))
    .withColumn("is_error", F.when(F.col("device_status") == "ERROR", 1).otherwise(0))

    # Handle nulls in anomaly_type column
    .withColumn("anomaly_type", F.when(F.col("anomaly_type").isNull(), "No
Anomaly").otherwise(F.col("anomaly_type")))
)

# Deduplicate based on OrderID
# DLT handles deduplication automatically if a primary key is defined in the pipeline settings.
return df_silver.dropDuplicates(["order_id"])

# -----
# Gold Layer: Aggregation (Final KPIs)
# -----

# These tables are created from the Silver layer and are ready for Power BI.

# KPI 1: Monthly Sales per Region
@dlt.table(
    name="tiasacatalog3.gold.monthly_sales_kpis",
    comment="Monthly sales per region for executive reporting."
)

```

```

)

def gold_monthly_sales():

    df_silver = dlt.read("tiasacatalog3.silver.iot_cleaned_silver") # Corrected

    return (df_silver

        .withColumn("order_month", F.trunc(F.col("order_time"), "month"))

        .groupBy("order_month", "region", "store_id")

        .agg(

            F.round(F.sum("sales_amount_calculated"), 2).alias("monthly_sales"),

            F.sum("anomaly_flag").alias("monthly_anomalies")

        )

    )

```

KPI 2: Store Tiers & Anomaly Impact

```

@dlt.table(

    name="tiasacatalog3.gold.store_performance_kpis",

    comment="Store tiers and anomaly impact on sales."

)

def gold_store_performance():

    df_silver = dlt.read("tiasacatalog3.silver.iot_cleaned_silver") # Corrected

    store_kpis = (df_silver

        .groupBy("store_id")

        .agg(

            F.sum("sales_amount_calculated").alias("total_sales"),

            F.sum("anomaly_flag").alias("total_anomalies"),

            F.count("order_id").alias("total_orders")

        )

    )

    return (store_kpis

```

```

.withColumn("store_tier",
    F.when(F.col("total_sales") > 10000, "High Performer")
    .when(F.col("total_sales") > 5000, "Medium Performer")
    .otherwise("Low Performer")
)
.withColumn("conversion_rate",
    F.when(F.col("total_orders") > 0, F.col("total_anomalies") / F.col("total_orders"))
    .otherwise(F.lit(None))
)
)

```

KPI 3: Weighted Average of Sales vs Anomalies

```

@dlt.table(
    name="tiasacatalog3.gold.weighted_metrics_kpis",
    comment="Aggregated sales and anomaly counts by device type."
)

def gold_weighted_metrics():
    df_silver = dlt.read("tiasacatalog3.silver.iot_cleaned_silver") # Corrected
    return (df_silver
        .groupBy("device_type")
        .agg(
            F.round(F.sum("sales_amount_calculated"), 2).alias("total_sales"),
            F.sum("anomaly_flag").alias("total_anomalies")
        )
        .withColumn("sales_per_anomaly",
            F.when(F.col("total_anomalies") > 0, F.col("total_sales") / F.col("total_anomalies"))
            .otherwise(F.lit(None))
        )
    )

```

)

Code for EventHub IOT stream:-

Parameters

storage_account = "tiasastore"

container = "tia"

file_path = "retail_iot_data.csv"

Credentials - Replace with actual values (or keep them in a secret scope)

client_id = "6bc323f8-6933-4a2e-830a-6ea453566f7e"

tenant_id = "5950e39e-81d1-45a1-8618-fe39a39b0448"

client_secret = "KAb8Q~F7mvoPwxSIUxo19qhCepvbAogaK73~acza"

Configurations for ABFS access (no mount needed in UC)

spark.conf.set(f"fs.azure.account.auth.type.{storage_account}.dfs.core.windows.net", "OAuth")

spark.conf.set(f"fs.azure.account.oauth.provider.type.{storage_account}.dfs.core.windows.net",

```
"org.apache.hadoop.fs.azurebfs.oauth2.ClientCredsTokenProvider")
```

```
spark.conf.set(f"fs.azure.account.oauth2.client.id.{storage_account}.dfs.core.windows.net", client_id)
```

```
spark.conf.set(f"fs.azure.account.oauth2.client.secret.{storage_account}.dfs.core.windows.net", client_secret)
```

```
spark.conf.set(f"fs.azure.account.oauth2.client.endpoint.{storage_account}.dfs.core.windows.net",
```

```
f"https://login.microsoftonline.com/{tenant_id}/oauth2/token")
```

```
# -----
```

```
# Build ABFS path directly (no /mnt needed)
```

```
# -----
```

```
abfs_path = f"abfss://{container}@{storage_account}.dfs.core.windows.net/{file_path}"
```

```
# -----
```

```
# Try reading the file to verify access
```

```
# -----
```

```
# Read raw file from ADLS
```

```
df = spark.read.format("csv").option("header", "true").load(abfs_path)
```

```
display(df)
```

```
import dlt
```

```
import pyspark.sql.functions as F
```

```
from pyspark.sql.types import StructType, StructField, StringType, IntegerType, DoubleType, TimestampType
```

```
# Define schema for the raw CSV data
```

```
iot_schema = StructType([
```

```
    StructField("OrderID", StringType(), True),
```

```
    StructField("CustomerID", StringType(), True),
```

```
    StructField("Region", StringType(), True),
```

```
    StructField("StoreID", StringType(), True),
```

```
    StructField("Product", StringType(), True),
```

```
    StructField("Quantity", IntegerType(), True),
```

```
    StructField("UnitPrice", DoubleType(), True),
```

```
    StructField("SalesAmount", DoubleType(), True),
```

```
StructField("OrderTime", StringType(), True),
```

```
StructField("DeviceType", StringType(), True),
```

```
StructField("Temperature", DoubleType(), True),
```

```
StructField("DeviceStatus", StringType(), True),
```

```
StructField("AnomalyFlag", IntegerType(), True),
```

```
StructField("AnomalyType", StringType(), True)
```

```
])
```

```
# -----
```

```
# Bronze Layer: Ingestion (Raw Data)
```

```
# -----
```

```
# This table ingests the raw data stream using Auto Loader.
```

```
@dlt.table(
```

```
    name="iot_orders_bronze",
```

```
    comment="Raw ingested data from ADLS."
```



```
)
```

```
def iot_orders_bronze():
```

```
    # Use spark.readStream with cloudFiles (Auto Loader) for a production-like streaming setup.
```

```
    return (
```

```
        spark.readStream
```

```
            .format("cloudFiles")
```

```
            .option("cloudFiles.format", "csv")
```

```
            .option("header", True)
```

```
            .schema(iot_schema)
```

```
            .load("abfss://tia@tiasastore.dfs.core.windows.net/incoming")
```

```
    )
```

```
EVENTHUB_CONNECTION_STRING =
```

```
"Endpoint=sb://tiasaeventhub.servicebus.windows.net/;SharedAccessKeyName=manage;SharedAccessKey=MWDy1/TV/ZLmpH2nWr9xbkBXRJGv8ZG7N+AEhAg6Ofg=;EntityPath=tiasa_retail_iot"
```

```
EVENT_HUB_NAME = "tiasa_retail_iot"
```

```

@dlt.table(

    name="iot_orders_bronze_eventhub",

    comment="Raw IoT data ingested from Event Hub"

)

def iot_orders_bronze_eventhub():

    # Read stream from Event Hub

    df = (spark.readStream

        .format("eventhubs")

        .option("eventhubs.connectionString", EVENTHUB_CONNECTION_STRING)

        .load()

    )

    # Parse Event Hub body (binary) as JSON

    df_parsed = (df.selectExpr("CAST(body AS STRING) as json_str")

        .select(F.from_json("json_str", iot_schema).alias("data"))

```

```

        .select("data.*")

    )

    return df_parsed

# -----

# Silver Layer: Transformation (Cleaned Data)

# -----

# This table cleans the Bronze data and performs type casting and deduplication.

@dlt.table(

    name="nilay_retail_iot_catalog.silver.iot_cleaned_silver",

    comment="Cleaned and validated data with correct types and no duplicates.",

    table_properties={"delta.enableChangeDataFeed": "true"} # Enable Change Data Feed for upserts

)

def iot_cleaned_silver():

    df_bronze = dlt.read("iot_orders_bronze")

```

```

df_bronze_eventhub = dlt.read("iot_orders_bronze_eventhub")

# Combine both sources using unionByName

df_combined = df_bronze.unionByName(df_bronze_eventhub, allowMissingColumns=True)

# Renaming columns for clarity and consistency

df_silver = df_combined.toDF(

    "order_id", "customer_id", "region", "store_id", "product", "quantity",

    "unit_price", "sales_amount", "order_time", "device_type", "temperature",

    "device_status", "anomaly_flag", "anomaly_type"

)

# Apply data type conversions and transformations

df_silver = (df_silver

    .withColumn("order_time", F.to_timestamp(F.col("order_time")))

    .withColumn("sales_amount_calculated", F.col("quantity") * F.col("unit_price"))

    .withColumn("is_error", F.when(F.col("device_status") == "ERROR", 1).otherwise(0))

```

```

# Handle nulls in anomaly_type column

    .withColumn("anomaly_type", F.when(F.col("anomaly_type").isNull(), "No
Anomaly").otherwise(F.col("anomaly_type")))

)

# Deduplicate based on OrderID

# DLT handles deduplication automatically if a primary key is defined in the pipeline settings.

return df_silver.dropDuplicates(["order_id"])

# -----

# Gold Layer: Aggregation (Final KPIs)

# -----

# These tables are created from the Silver layer and are ready for Power BI.

# KPI 1: Monthly Sales per Region

@dlt.table(

    name="tiasa_retail_iot_catalog.gold.monthly_sales_kpis",

    comment="Monthly sales per region for executive reporting."

```

```
)
```

```
def gold_monthly_sales():
```

```
    df_silver = dlt.read("tiasa_retail_iot_catalog.silver.iot_cleaned_silver") # Corrected
```

```
    return (df_silver
```

```
        .withColumn("order_month", F.trunc(F.col("order_time"), "month"))
```

```
        .groupBy("order_month", "region", "store_id")
```

```
        .agg(
```

```
            F.round(F.sum("sales_amount_calculated"), 2).alias("monthly_sales"),
```

```
            F.sum("anomaly_flag").alias("monthly_anomalies")
```

```
        )
```

```
    )
```

```
# KPI 2: Store Tiers & Anomaly Impact
```

```
@dlt.table(
```

```
    name="tiasa_retail_iot_catalog.gold.store_performance_kpis",
```

```
comment="Store tiers and anomaly impact on sales."
```

```
)
```

```
def gold_store_performance():
```

```
    df_silver = dlt.read("tiasa_retail_iot_catalog.silver.iot_cleaned_silver") # Corrected
```

```
    store_kpis = (df_silver
```

```
        .groupBy("store_id")
```

```
        .agg(
```

```
            F.sum("sales_amount_calculated").alias("total_sales"),
```

```
            F.sum("anomaly_flag").alias("total_anomalies"),
```

```
            F.count("order_id").alias("total_orders")
```

```
        )
```

```
    )
```

```
    return (store_kpis
```

```
        .withColumn("store_tier",
```

```

F.when(F.col("total_sales") > 10000, "High Performer")

.when(F.col("total_sales") > 5000, "Medium Performer")

.otherwise("Low Performer")

)

.withColumn("conversion_rate",

F.when(F.col("total_orders") > 0, F.col("total_anomalies") / F.col("total_orders"))

.otherwise(F.lit(None))

)

)

# KPI 3: Weighted Average of Sales vs Anomalies

@dlt.table(

name="tiasa_retail_iot_catalog.gold.weighted_metrics_kpis",

comment="Aggregated sales and anomaly counts by device type."

)

```



```

def gold_weighted_metrics():

    df_silver = dlt.read("tiasa_retail_iot_catalog.silver.iot_cleaned_silver") # Corrected

    return (df_silver

        .groupBy("device_type")

        .agg(

            F.round(F.sum("sales_amount_calculated"), 2).alias("total_sales"),

            F.sum("anomaly_flag").alias("total_anomalies")

        )

        .withColumn("sales_per_anomaly",

            F.when(F.col("total_anomalies") > 0, F.col("total_sales") / F.col("total_anomalies"))

            .otherwise(F.lit(None))

        )

    )

```

Microsoft Azure

Search resources, services, and docs (G+)

Copilot

tredecence22@michaelklo...
DEFAULT DIRECTORY (MICHAELK...

Home > Event Hubs > tiasaeventhub > eventhubtiasa (tiasaeventhub/eventhubtiasa)

eventhubtiasa (tiasaeventhub/eventhubtiasa) | Shared access policies

Event Hubs Instance

Search

◊ << + Add

Search to filter items...

Overview

Access control (IAM)

Diagnose and solve problems

Data Explorer

Resource visualizer

Settings

Shared access policies ☆

Configuration

Properties

Locks

Entities

Features

Automation

Help

Policy

manage

SAS Policy: manage

Event Hub

Event Hubs access control with Shared Access Signatures [Learn more](#)

Claims

☒ Manage

☒ Send

☒ Listen

Primary key

.....

Secondary key

.....

Primary connection string

.....

Secondary connection string

.....

SAS policy ARM ID

/subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/resourceGroups/LabsKra...

☐ Show AMQP connection strings

Save Discard changes

Give feedback

https://portal.azure.com/#@michaelkoudkrafthotmail.onmicrosoft.com/resource/subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/resourceGroups/LabsKra.../providers/Microsoft.EventHub/namespaces/tiasaeventhub/e...